CODING TECHNICAL ASSESSMENT

Position: Full Stack Java Developer

Level: Mid-level

Technologies: Java, HTML, CSS, JavaScript

INSTRUCTIONS:

• This is a coding-focused technical assessment
• Complete all coding exercises and projects
• Submit your code files with clear documentation
• Time limit: 2-3 hours
• Reply to this email with your completed solutions

Full Stack Java Developer Mid-level Coding Assessment

Time Limit: 3 hours

Instructions:

1. Create a new directory for the assessment and add all your solutions to it.
2. Use Java 8 or later for Java-related tasks.
3. Use HTML, CSS, and JavaScript for front-end tasks.
4. Use a Java-based database like H2 or Derby for database-related tasks.
5. Follow standard naming conventions for classes, methods, and variables.
6. Ensure your code is readable, maintainable, and follows best practices.
7. Include a README file with instructions on how to run and test your code.

Submission Guidelines:

1. Zip the assessment directory and submit it via email.
2. Ensure the zip file is named as `FullStackJavaDeveloperAssessment.zip`.
3. Include your name and contact information in the README file.

Coding Exercises (7 questions)

Question 1: Algorithm - Find the First Duplicate in an Array (15 minutes)

* Problem Statement: Find the first duplicate in an unsorted array of integers.
* Input/Output Examples:
’² –ç WC `[2, 1, 3, 5, 3, 2]`
’² ÷WG WC `3`
* Evaluation Criteria:
’² 6÷'&V7FæW70
’² Vff–6–Væ7•
* Sample Test Cases:
’² ³"Â Â 2Â RÂ 2Â %Ö Óâ 6
’² ³ Â "Â 2Â BÂ UÖ Óâ Ó †æò GW Æ–6 FW2•

Question 2: Code Implementation - Java Bean Validation (20 minutes)

* Problem Statement: Implement Java Bean Validation for a given User class.
* Input/Output Examples:
’² –ç WC `User user = new User("John", "Doe", "johndoe@example.com");`
’² ÷WG WC `Valid user data`
* Evaluation Criteria:
’² 6÷'&V7FæW70
’² 6öFR V Æ—G•
* Sample Test Cases:
’² W6W" W6W" Ò æPw User("John", "Doe", "johndoe@example.com");` => `Valid user data`
’² W6W" W6W" Ò æPw User("", "", "invalid email");` => `Invalid user data`

Question 3: Bug Fixing - Java Multithreading (20 minutes)

* Problem Statement: Fix the given Java multithreading code to avoid deadlocks.
* Input/Output Examples:
’² –ç WC `Thread thread = new Thread(new DeadlockThread());`
’² ÷WG WC `No deadlocks detected`
* Evaluation Criteria:
’² 6÷'&V7FæW70

’² 6öFR V Æ—G•
* Sample Test Cases:
’² F‡&V B F‡&V B Ò æPw Thread(new DeadlockThread());` => `No deadlocks detected`
’² F‡&V B F‡&V B Ò æPw Thread(new DeadlockThread()); thread.start();` => `Deadlock detected`

Question 4: Code Optimization - Java String Manipulation (15 minutes)

* Problem Statement: Optimize the given Java string manipulation code for better performance.
* Input/Output Examples:
’² –ç WC `String str = "Hello World";`
’² ÷WG WC `Optimized string manipulation`
* Evaluation Criteria:
’² erformance
’² 6öFR V Æ—G•
* Sample Test Cases:
’² 7G ing str = "Hello World";` => `Optimized string manipulation`
’² 7G ing str = "Very long string";` => `Optimized string manipulation`

Question 5: API Development - RESTful API (30 minutes)

* Problem Statement: Develop a RESTful API to manage users.
* Input/Output Examples:
’² –ç WC `GET /users`
’² ÷WG WC `[{"id":1,"name":"John Doe","email":"johndoe@example.com"}]`
* Evaluation Criteria:
’² 6÷'&V7FæW70
’² 6öFR V Æ—G•
* Sample Test Cases:
’² tUB ÷W6W'6 Óâ ·²&–B#£ Â&æ ÖR#¢$¦ö†â FöR"Â&VÖ –Â#¢&¦ö†æFöT Pxample.com"}]`
’²   õ5B ÷W6W'6 Óâ W6W" 7&V FVB 7V66W76gVÆÇ–

Question 6: Database Query - SQL (20 minutes)

* Problem Statement: Write a SQL query to retrieve the top 5 most active users.
* Input/Output Examples:
'² –ç WC `SELECT * FROM users ORDER BY activity DESC LIMIT 5;`
'² ÷WG WC `[{"id":1,"name":"John Doe","email":"johndoe@example.com","activity":10}]`
* Evaluation Criteria:
'² 6÷'&V7FæW70
'² Vff–6–Væ7•
* Sample Test Cases:
'² 4TÄT5B ¢ e OM users ORDER BY activity DESC LIMIT 5;` => `[{"id":1,"name":"John Doe","email":"johndoe@example.com","activity":10}]`
'² 4TÄT5B ¢ e OM users ORDER BY activity ASC LIMIT 5;` => `[{"id":5,"name":"Jane Doe","email":"janedoe@example.com","activity":1}]`

Question 7: Front-end - HTML/CSS/JavaScript (30 minutes)

* Problem Statement: Create a simple login form using HTML, CSS, and JavaScript.
* Input/Output Examples:
'² –ç WC `username: john, password: doe`
'² ÷WG WC `Login successful`
* Evaluation Criteria:
'² 6÷'&V7FæW70
'² 6öFR V Æ—G•
* Sample Test Cases:
'² W6W name: john, password: doe` => `Login successful`
'² W6W name: invalid, password: invalid` => `Invalid credentials`

Practical Projects (2 mini-projects)

Project 1: To-Do List App (45 minutes)

* Problem Statement: Create a simple to-do list app using Java and HTML/CSS/JavaScript.
* Evaluation Criteria:

’² 6÷'&V7FæW70
’² 6öFR V Æ—G•
’² W6W" Pxperience
* Sample Test Cases:
’²   FB æPw task` => `Task added successfully`
’² FVÆWFR F 6¶ Óâ @ask deleted successfully`

Project 2: Simple Chat App (45 minutes)

* Problem Statement: Create a simple chat app using Java and HTML/CSS/JavaScript.
* Evaluation Criteria:
’² 6÷'&V7FæW70
’² 6öFR V Æ—G•
’² W6W" Pxperience
* Sample Test Cases:
’² 6VæB ÖW76 vV Óâ ÖW76 vR 6VçB 7V66W76gVÆÇ–
’² &V6V—`e message` => `Message received successfully`

Code Review Tasks (2 questions)

Question 1: Code Review - Java (20 minutes)

* Problem Statement: Review the given Java code and suggest improvements.
* Input/Output Examples:
’² –ç WC `Java code`
’² ÷WG WC `Improved Java code`
* Evaluation Criteria:
’² 6÷'&V7FæW70
’² 6öFR V Æ—G•
* Sample Test Cases:
’²   ava code` => `Improved Java code`

Question 2: Code Refactoring - Java (20 minutes)

* Problem Statement: Refactor the given Java code for better performance.
* Input/Output Examples:
’² –ç WC `Java code`
’² ÷WG WC `Refactored Java code`
* Evaluation Criteria:
’² erformance
’² 6öFR V Æ—G•
* Sample Test Cases:
’²   ava code` => `Refactored Java code`

Generated on: 15/09/2025 14:38:38